

FLEXBMS

System Overview

Home battery energy storage system

Document date	20 August 2026
System	96s LiFePO4 home BESS
Nominal capacity	314 Ah / approximately 96.5 kWh
Primary controller	STM32G491
Document status	Living architecture overview

This document explains the current FlexBMS system at architecture level. Schematics and source code remain authoritative for implementation details.

Contents

1. System level	3
1.1. System roles	3
1.2. Energy and information flow	3
2. Hardware	3
2.1. Battery assembly	3
2.2. Module boards	4
2.3. Master controller	4
2.4. High-voltage path	4
2.5. Physical communications	5
3. Software	5
3.1. STM32	5
3.2. HV supervisor	9
3.3. ESP32	9
3.4. Energy management system	15
3.5. Home Assistant integration	15
3.6. Companion application	16
4. Operating concept	16
4.1. Start-up	16
4.2. Normal operation	16
4.3. Faulted operation	17
5. Interfaces and ownership	17
6. Current project structure	17
7. Open topics	18

1. System level

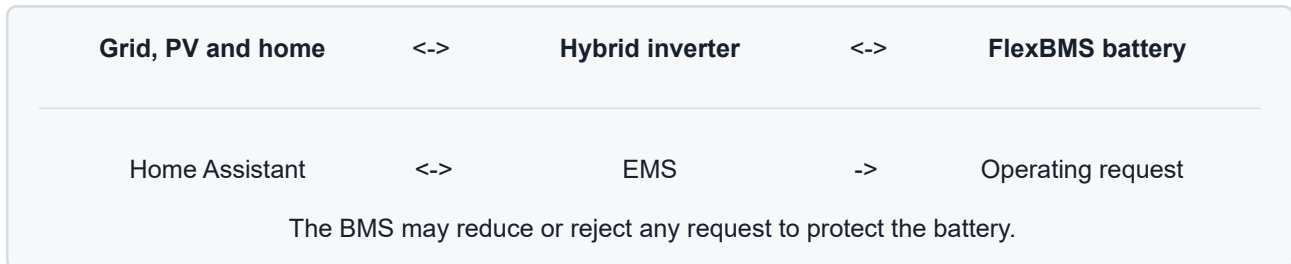
FlexBMS is the battery management and integration platform for a stationary home battery energy storage system. The battery consists of 96 LiFePO₄ cells in series, arranged as four 24-cell modules. At a nominal cell voltage of 3.2 V and a capacity of 314 Ah, the pack is approximately 307 V and 96.5 kWh.

The system connects the battery to a three-phase hybrid inverter. FlexBMS measures the battery, protects it, controls the high-voltage connection, and provides the information needed by higher-level energy management. The battery management system remains the final authority for battery safety.

1.1. System roles

Layer	Responsibility	Status
Battery and HV hardware	Stores energy, measures physical quantities, and safely connects the pack to the inverter.	CURRENT
STM32 BMS	Performs real-time measurement, protection, state handling, and contactor control.	CURRENT
ESP32 gateway	Provides local BMS maintenance, Wi-Fi provisioning, and telemetry transport while remaining listen-only on CAN.	CURRENT
Energy management system	Chooses when and how much to charge or discharge within BMS limits.	PLANNED
Home Assistant	Provides dashboards, automation, and a user-facing view of the installation.	CURRENT

1.2. Energy and information flow



Control principle

Energy management is advisory; battery protection is mandatory. The EMS may request an operating point, but the BMS defines the permitted charge and discharge envelope and can open the contactor.

2. Hardware

2.1. Battery assembly

The pack contains 96 series-connected LiFePO₄ cells divided into four physical 24s modules. Each module contains two twelve-cell monitoring sections. A 200 A / 75 mV shunt is located at the negative end of the pack for current measurement.

Item	Current baseline
Cell chemistry	LiFePO4
Series configuration	96s
Physical modules	4 x 24s
Cell capacity	314 Ah
Nominal pack voltage	307.2 V
Nominal stored energy	Approximately 96.5 kWh
Current shunt	200 A / 75 mV

The modules are installed in an unheated garage. Temperature measurement is distributed across the pack. The current STM32 configuration expects four NTC inputs per battery-monitor IC. Final sensor population and harness documentation should be kept consistent with the assembled modules.

2.2. Module boards

Each 24s module has one FlexBMS module board with two MC33771C battery cell controllers. Each controller measures twelve cells, supports passive balancing, reads local temperature sensors, and participates in the trans-former-isolated TPL daisy chain.

Across the complete pack this gives:

- eight MC33771C devices;
- 96 monitored cell voltages;
- up to 32 configured NTC measurements; and
- one current-sensing location on the first, lowest-voltage controller.

2.3. Master controller

The master board is the central interface between the battery, high-voltage switching hardware, service tools, inverter side, and local network.

Its main functional blocks are:

- STM32G491 real-time controller;
- MC33664 TPL transceiver for the module-board chain;
- ESP32-C3-WROOM-02U connectivity controller;
- isolated CAN and RS485 interfaces;
- isolated measurements of battery-side and load-side high voltage;
- contactor and precharge relay drivers;
- USB for flashing, diagnostics, and the Companion application; and
- power selection between USB and the battery-derived low-voltage supply.

The board contains separate HV, LV, and inverter-side isolated domains. USB-only power supports development and diagnostics, but does not provide the 12 V rail required to energize the contactor.

2.4. High-voltage path

The main contactor is located in the positive battery path. A separate precharge path limits inrush current before the contactor closes. The STM32 is the sole software owner of both drivers. It measures the battery and load sides through separate isolated AMC3330 channels, compares the battery-side result with the voltage reported by the cell-monitor chain, and observes both active-low diagnostic outputs.

The contactor is pulled in at full drive and then held with 20 kHz PWM. Precharge remains closed during pull-in and opens only after the contactor drive is established. The hold duty is a firmware configuration value.

Voltage feedback shows whether the two HV nodes are plausible and whether precharge has progressed. It does not prove the mechanical position of the contactor: the load-side voltage may remain present after opening because the inverter DC-link discharge behavior is not yet characterized.

Pack-level protection also includes the main DC fuse and a manual DC isolation device. Exact component ratings, wiring, creepage, clearance, and protection coordination belong in the hardware design files and commissioning documentation.

2.5. Physical communications

Interface	Connection	Purpose
TPL	Master to 8 monitor ICs	Cell, temperature, diagnostic, and current data.
USB	Service computer to STM32	Firmware, logs, and Companion access.
CAN	STM32 and inverter-side domain	Battery status and inverter communication.
Wi-Fi	ESP32 to local network	Local maintenance page, Wi-Fi provisioning, and future telemetry integration.
RS485	ESP32 to external energy meters	Optional read-only Modbus RTU polling of up to two SDM72D-M-2 meters.

3. Software

The software is divided by responsibility. Time-critical protection and actuation remain on the STM32. Network-facing functions belong on the ESP32. Site optimization belongs in the EMS, with Home Assistant providing the local user and automation layer.

3.1. STM32

CURRENT

The STM32 firmware is the real-time BMS. It starts and monitors the MC33771C chain, gathers a complete battery measurement set, evaluates battery and communication faults, and controls relay permission.

The current battery profile is:

- eight MC33771C devices with twelve cells each;
- four NTC channels per controller on GPIO0 through GPIO3;
- current sensing on CID 1 with a 375 micro-ohm shunt value;
- positive current for charging and negative current for discharging;
- 314 Ah nominal capacity; and
- amp-hour retention in RTC backup registers while backup power is present.

Static firmware settings contain the safety limits, monitor hardware template, and timing parameters. Companion-configurable values such as the shunt resistance, capacity, current polarity, balancing gate, diagnostics gate, and monitor topology are kept in the versioned STM32 runtime configuration in flash; they are not duplicated in the static settings.

The firmware separates active fault conditions and acknowledgement-required latches. The HV supervisor receives permission only while the BMS is running without blocking faults and holds one fresh, complete measurement set from the monitor chain.

Runtime faults keep measurement and monitoring active while preventing relay closure. An acknowledgement is accepted only after the underlying runtime condition has disappeared.

The STM32 also hosts:

- the dedicated HV supervisor;
- board-level analog and digital I/O;
- CAN communication;
- USB serial and the trusted text command/debug console; and
- the framed BMS v1 service interface over isolated UART and USB CDC.

3.1.1. Todo:

High priority:

- Go through CubeMX configuration. Pinout is OK.
- Analog measurement of HV side. Including diagnostic pin checking. ADC configuraion, high accuracy low Fs.
- Precharge handling. Including:
- Contactor handling. Including: economizing, failure checking using voltages (to be discussed)
- USB VSENSE setup
- Derating on temperature (mainly low temperature charging, high temperature is not that applicable)

- GoodWe CAN communication. The current research and candidate protocols are documented below; implementation and empirical validation remain pending.
- User LED status language is specified in `Documentation/architecture/home-bess-firmware-and-maintenance.md`; firmware implementation remains pending.

Medium priority:

- Aux relay control (potentially useful for a future heating system).

3.1.2. GoodWe inverter CAN protocol research

RESEARCH / NOT IMPLEMENTED

This section records the GoodWe CAN findings available before implementation. It is deliberately an evidence-weighted candidate specification, not yet a confirmed contract for the GW12K-ET-20. The target inverter is the high-voltage ET-20 model. The present CAN implementation is not a compatibility constraint: the STM32 CAN peripheral, bitrate, identifiers, and payload handling can be changed as part of this work. The CAN bus is expected to contain only the STM32 and the inverter. The ESP32 Gateway remains listen-only.

Important distinction

Two different families are being mixed in community material. A GoodWe high-voltage candidate uses 250 kbit/s and identifiers in the 0x453–0x460 range. A separate SMA/Pylontech-style profile uses 500 kbit/s and identifiers such as 0x351, 0x355, 0x356, and 0x359. The latter is not evidence that the former is used by the GW12K-ET-20.

3.1.2.1. Evidence status

Source	Relevant finding	Assessment
GoodWe GW6000-EH HV1 document	Reverse-engineered, non-official document. It specifies 11-bit CAN at 250 kbit/s, one-second-class updates, 0x453–0x458, and additional 0x45A/0x460 frames. It also describes the inverter timeout frame 0x420 containing GWBATHV.	Strongest public HV lead, but different model and not official.
ai-republic discussion 19	A GW5000-ES-20 user reported that newer G2 firmware rejected DIY BMS emulators. The author said a GoodWe HV specification worked for the GW6000-EH, but initially had no binding. The discussion does not prove an ET-20 frame map.	Important firmware/profile warning.
Current ai-republic HV descriptor and processor	The current code selects 250 kbit/s and sends 0x453, 0x455, 0x456, 0x457, and 0x458. It does not parse an inbound request and does not send the 0x45A or 0x460 frames listed by the reverse-engineered document.	Useful implementation lead; not target validation.
syssi discussion 114	The working trace uses the SMA/Pylontech-style family: 500 kbit/s, 0x359, 0x351, 0x355, 0x356, and 0x354, with an inverter response at 0x305. A later report says stale charge-disable state made the GoodWe unhappy.	Useful LV/SMA evidence, not HV ET evidence.

No official GoodWe public document found so far provides the complete BMS CAN frame map for the GW12K-ET-20. The official battery compatibility information establishes product and battery compatibility, but does not settle the bitrate, identifiers, payload byte order, or firmware-specific handshake.

3.1.2.2. Candidate A: GoodWe HV protocol, highest current probability

This is the first protocol I would investigate for the GW12K-ET-20. The confidence is based on the model family, the independent HV document, and the current GoodWe HV sender implementation. It remains a candidate until the actual inverter traffic is captured or the inverter accepts it in a controlled test.

ID	Direction	Candidate payload	Confidence
0x420	Inverter -> STM32	Timeout/no-BMS beacon. ASCII GWBATHV is 47 57 42 41 54 54 48 56. Approximately one-second-class timing is reported; one copy states 1.3 s.	Medium for HV family; unconfirmed for ET-20.

0x425	Inverter -> STM32	Inverter-measured battery voltage and current. The public document says negative current means charging; the remaining bytes are not decoded.	Low/medium.
0x453	STM32 -> inverter	Number of 48 V battery modules. The document proposes integer nominal pack voltage divided by 48; that would produce a candidate value of 6 for this approximately 307 V pack.	Medium.
0x455	STM32 -> inverter	Two little-endian 16-bit bitmaps: BMS alarms followed by BMS warnings. Exact bit semantics remain provisional.	Medium.
0x456	STM32 -> inverter	Four little-endian 16-bit values: charge-voltage limit, charge-current limit, maximum discharge current, and minimum discharge voltage. Voltage/current resolution is 0.1 V/0.1 A.	Medium/high.
0x457	STM32 -> inverter	SoC and SoH as little-endian 16-bit values with 0.01% resolution, followed by four reserved bytes.	Medium/high.
0x458	STM32 -> inverter	Actual pack voltage, battery current, and average battery temperature as little-endian 16-bit values with 0.1 V, 0.1 A, and 0.1 °C resolution, followed by two reserved bytes.	Medium/high.
0x45A	STM32 -> inverter	Eight zero bytes in the reverse-engineered table. The document associates this frame with an inverter response at 0x425.	Low/medium; validate before use.
0x460	STM32 -> inverter	Request flags related to enabling battery charging/discharging. The public document does not decode the flags.	Low; validate before use.

For this candidate, the actual-voltage field in 0x458 should use the measured pack voltage selected for FlexBMS, not a voltage inferred from SoC. The charge and discharge limits in 0x456 should be generated from fresh BMS limits. A loss of valid BMS data must never result in a larger permitted limit; the exact conservative fallback and whether a zero limit is accepted by this GoodWe firmware must be established during testing.

The current ai-republic sender is useful because it confirms the field layout above in executable code, but it is not a complete specification. In particular, it omits 0x45A and 0x460, does not parse 0x425, and its warning-building code should be reviewed before reuse.

3.1.2.3. Candidate B: SMA/Pylontech-style CAN protocol, fallback only

This is the protocol described by the ESPHome/JK-BMS community work. It is a credible alternative only if the ET-20 firmware is configured for that battery profile. It should not be called the GoodWe HV protocol without evidence.

ID	Direction	Candidate payload
0x351	STM32 -> inverter	Charge-voltage limit, charge-current limit, discharge-current limit, and discharge-voltage limit; the published SMA-style mapping uses 0.1 V/0.1 A values.
0x355	STM32 -> inverter	SoC, SoH, and optional high-resolution SoC.
0x356	STM32 -> inverter	Actual battery voltage, current, and average temperature. The current is signed in the usual mapping, with positive commonly representing discharge.
0x359	STM32 -> inverter	Alarm and warning/status information. Exact bit interpretation must follow the selected SMA/Pylontech profile.
0x354	STM32 -> inverter	Present in the community trace, but its exact payload is not part of the current FlexBMS candidate contract.
0x305	Inverter -> STM32	Observed by the community implementation as an inverter response/ACK. This must not be assumed to be the HV handshake.

The evidence for Candidate B points to 500 kbit/s and a different byte-level contract from Candidate A. It is retained as a fallback because GoodWe G2 firmware and battery profiles may change the accepted protocol. It is not the first implementation target for the high-voltage ET-20 inverter.

3.1.2.4. Implemented first software slice

IMPLEMENTED / NOT HARDWARE-VALIDATED

The STM32 now selects the protocol through `Software/Master/Inc/Peripherals/GoodweCanConfig.h`. Candidate A is the default and selects 250 kbit/s; Candidate B selects 500 kbit/s. The CubeMX default and generated FDCAN timing use Candidate A, while the startup code applies the selected timing before starting the shared CAN driver.

The transmitter publishes the core frames once per second. Candidate A sends 0x453, 0x455, 0x456, 0x457, 0x458, the eight-zero-byte 0x45A diagnostic request, and the two-byte 0x460 default- battery compatibility frame with payload 00 00. The latter was enabled for the inverter's `Default *6` profile after the core frames were accepted but the inverter continued to report BMS: Discharge disabled. 0x45A is enabled to request the inverter's 0x425 measured voltage/current response during the diagnostic test; that response remains observation-only. Candidate B sends 0x351, 0x355, 0x356, and 0x359. Candidate B 0x354 remains disabled because its use is not sufficiently confirmed.

The CAN receiver passively records standard frames 0x420, 0x425, and 0x305, including counters, timestamps, and the last payload. These frames do not receive application-level replies. The CAN peripheral handles the normal link-layer acknowledgement automatically. The reverse-engineered Candidate A material describes 0x420 as a timeout/no-BMS indication, not as a confirmed request requiring a response. The Candidate B material describes 0x305 as an inverter-originated periodic frame; community testing also treated it as an ACK or keepalive rather than a BMS request.

The STM32 publishes a 500 ms GoodWe diagnostic snapshot to Companion through the normal framed UART/Gateway path. Companion's Diagnostics page shows every Candidate A transmit frame's FDCAN-queue acceptance count and last-accept time, skipped cycles caused by an unavailable coherent BMS snapshot, the total and most recent transmit failure, the exact voltage/current last sent in 0x458, and the raw 0x420, 0x425, and 0x305 receive counters, timestamps, and payloads. It also shows the hardware transmit/receive error counters, protocol state, bus-off/warning/error-passive flags, HAL error code, and TX FIFO free level so queue acceptance is not mistaken for confirmation of a completed CAN transmission. Browser-local CSV logging includes the same data, plus a clearly labelled candidate 0.1-unit decode of 0x425; this decode is diagnostic evidence and not an input to battery safety or inverter limits.

Application-level responses are guarded by `GOODWE_CAN_ENABLE_APPLICATION_RESPONSES`, which defaults to 0. It must not be enabled until a capture from the actual inverter identifies a required response and its payload.

CAN data is published only from a coherent BMS snapshot. Stale or invalid electrical or thermal measurements stop transmission; an unavailable SoC does not. A fresh snapshot with a common fault keeps the status frames alive but forces both directional current limits to zero. The configured under-temperature threshold is charge-only: it sets the charge current limit to zero while leaving discharge available if all common safety conditions remain healthy. For the EVE MB31 LiFePO4 pack, the inverter-facing charge target is fixed at 3.500 V per cell and charge current derates linearly from the configured maximum at 3.400 V maximum-cell voltage to zero at 3.500 V. The discharge-voltage target is 2.900 V per cell and discharge current derates linearly from the configured maximum at 2.900 V minimum-cell voltage to zero at 2.800 V. Lower current limits take effect immediately; increases recover at 10% of the configured maximum per second. These are soft, directional inverter limits and do not replace or latch the independent 3.600 V overvoltage and 2.500 V undervoltage BMS trips. Pack voltage uses the BCC cell-voltage sum. The Candidate A module count defaults to six and SoH defaults to 100% until a real SoH source exists. If no retained SoC calibration is available, the transmitter still publishes the coherent measurements and advertises 30% SoC until the automatic full-charge calibration completes; after that it publishes the calibrated SoC.

3.1.2.5. Open protocol questions

- Does the GW12K-ET-20 accept the 250 kbit/s GoodWe HV profile, and does its current ARM/inverter firmware require a particular battery profile?
- Is 0x420 only a timeout indication, or does this firmware require an additional response? The available HV document does not establish a reply to 0x420; the implemented receiver records it so this can be verified.
- Does the 0x460 00 00 frame clear the discharge-disable status on the GW12K-ET-20 with the `Default *6` battery profile? Is 0x45A also required, and what values and timing do approved batteries use?
- Does the inverter emit 0x425 after receiving 0x45A, and can that frame be used for diagnostics without becoming a BMS safety input?
- What are the exact signs, limits, alarm bits, and timeout intervals on this model and firmware?
- Does the inverter require all frames at a fixed one-second period, and does it tolerate a burst of frames or require spacing between them?

3.1.2.6. Validation boundary

The implemented software is a protocol-specific STM32 transmitter with passive receive diagnostics. The STM32 remains the safety authority: inverter-facing CAN frames communicate measured state and permitted limits, but they do not replace BMS fault evaluation, contactor permission, or HV supervision. Home Assistant may request an operating point through the normal control path, but it must not write arbitrary CAN payloads.

The next evidence needed is a passive CAN capture from the actual GW12K-ET-20 and its installed battery/firmware, ideally covering startup, normal operation, charge permission removal, discharge permission removal, and an inverter timeout. Until that exists, Candidate A is the best starting hypothesis, while Candidate B remains a deliberately documented fallback.

3.2. HV supervisor

CURRENT

The HV supervisor follows OFF -> SELF_TEST -> PRECHARGE -> CONTACTOR_CLOSE -> RUN. The transport-independent `setRunRequest(bool)` operation is implemented through the framed BMS v1 named service. It provides intent only; it does not command a contactor. The STM32 accepts operation only when its BMS and HV-supervisor conditions are healthy.

SELF_TEST checks BMS permission, measurement freshness, isolated-voltage diagnostics, agreement between the battery-side and cell-monitor voltages, an unenergized load side, and the available power source. PRECHARGE raises the load-side voltage and requires a stable voltage relationship before contactor pull-in. CONTACTOR_CLOSE transfers from the precharge path to contactor hold drive. RUN keeps monitoring the BMS permission and both HV measurements.

A removed request, lost BMS permission, or implausible HV feedback disables both drivers. A blocking fault does not erase a true run request: after an explicit user acknowledgement and successful STM32 revalidation, the BMS may resume the startup sequence. The request defaults off after an STM32 reset; loss of a Gateway or USB Companion transport does not alter it. The STM32 reports requested state, actual HV state, and any blocking reason separately.

USB-only operation supports service access but cannot energize the HV path. A recorded run request still requires healthy BCC and HV validation before the STM32 can start. The fitted hardware pull-down defines the inactive USB-sense level; the STM32 does not enable an internal pull.

Source of truth

Thresholds, timings, fault definitions, CAN identifiers, and register-level behavior are intentionally not duplicated here. The STM32 source code and hardware schematics are authoritative.

3.3. ESP32

CURRENT

The ESP32 is the network and telemetry controller in the isolated inverter-side domain. Its intended role is to observe BMS and inverter information, publish selected data to the local network, and support integration with the EMS and Home Assistant.

The ESP32 is not part of the battery safety chain. It is not allowed to transmit commands on the BMS CAN bus and must remain listen-only there. Loss of the ESP32 or Wi-Fi must not prevent the STM32 from protecting or isolating the battery.

Software/Gateway implements isolated UART v1, Wi-Fi provisioning/station/recovery, and the local compiled Companion maintenance page. The Companion provides stale-status diagnostics, fresh BMS monitoring, CSV logging, register reads, Gateway NTP RTC synchronisation and device-time readback, fault clear, immediate run-request actions, and station-LAN firmware updates. MQTT Home Assistant discovery is a current Gateway function; CAN observation remains a separate planned increment. The Gateway uses isolated UART to the STM32 and remains listen-only on shared BMS/GoodWe CAN. It is not a safety authority. The detailed transport, local-network, and recovery design is in `Documentation/architecture/home-bess-firmware-and-maintenance.md`.

After a station reconnect, the Gateway withdraws and re-announces `flexbms.local`; a temporary mDNS failure is retried. Gateway OTA boots the inactive slot as pending and confirms it only after the local HTTP/WebSocket service starts and a station address remains stable for ten seconds. Saved credentials are attempted directly first; bounded

station retries continue even while the recovery AP is active. If stable connectivity cannot be established within three minutes, the ESP-IDF bootloader returns to the previous slot. The attempted version, rollback reason, and last Wi-Fi disconnect reason are retained for Companion diagnostics. Companion compares a content-derived web bundle identity and reloads its non-cacheable HTML shell automatically when a new bundle responds.

The local web service is intentionally bounded: it retains TCP capacity for MQTT and allows one queued telemetry frame per WebSocket. A browser that stops reading is retired after its first failed asynchronous send; a delayed callback from an old Companion socket cannot disturb its replacement connection.

The 4 MiB ESP32-C3 partition table uses two 1.5 MiB OTA application slots, a 512 KiB `bms_update` staging partition, and 384 KiB LittleFS reserved for future diagnostics/metadata. The committed Companion bundle is currently compiled into each application image, not served from LittleFS.

The Gateway application is written at the explicit `ota_0` offset in `Software/Gateway/partitions.csv` (currently `0x20000`), not ESP-IDF's generic `0x10000` default. Gateway release and normal PlatformIO upload tooling derive that address from the partition table; release creation verifies an ESP image magic byte at the derived address in the merged factory image.

3.3.1. Framed BMS protocol v1 contract

AGREED

The canonical byte-level implementation contract is `Documentation/protocol/uart-v1.md`. The following overview is retained in the system document; resolve any discrepancy in favour of the Markdown contract.

The STM32G491 BMS and ESP32-C3 Gateway communicate over isolated USART1 at 1 Mbit/s, 8-N-1, full duplex, without hardware flow control. Direct USB CDC uses the same framed bytes for the Web Serial Companion while retaining the trusted text command/debug console through a byte dispatcher. The STM32 is the safety authority. UART loss is a communication-health condition only: it must not force the HV path off or change the current run request. The `#!` BMS Companion protocol is retired.

Each endpoint sends a zero-payload heartbeat every 500 ms. A peer is declared lost only after 1.5 s without a complete CRC-valid frame. Invalid bytes or frames with a bad CRC do not refresh the timer. The Gateway and direct USB Companion share one STM32 service slot; a second request receives BUSY rather than being queued.

All multi-byte values are little-endian. Encoders and decoders write and read individual fields; they must not transmit native C or C++ structs. The maximum payload length is 512 bytes.

3.3.1.1. Frame

offset	size	field
0	2	MAGIC = 0x46 0x42 ("FB")
2	1	VERSION = 0x01
3	1	TYPE
4	1	SEQUENCE
5	2	LENGTH, uint16 payload length
7	n	PAYLOAD
7+n	4	CRC32, uint32

CRC is CRC-32/ISO-HDLC: polynomial `0x04C11DB7` (reflected `0xEDB88320`), reflected input and output, initial value and final xor `0xFFFFFFFF`. It covers the magic, header, and payload, but not the CRC field; the transmitted CRC word is little-endian. The standard check value is `CRC32("123456789") = 0xCBF43926`. On an invalid version, length, or CRC, the receiver discards the first magic byte and resumes magic scanning. The protocol uses no escaping or byte stuffing.

SEQUENCE = 0 is reserved for heartbeat, telemetry, and events. A Gateway or USB Companion service request uses a value from 1 through 255; the STM32 response echoes that sequence.

3.3.1.2. Types

ID	Type	Direction
0x01	HEARTBEAT	both
0x02	STATUS	STM32 to Gateway or USB Companion
0x03	PACK	STM32 to Gateway or USB Companion
0x04	CELL	STM32 to Gateway or USB Companion
0x05	TEMPERATURE	STM32 to Gateway or USB Companion

ID	Type	Direction
0x06	HV_VOLTAGES	STM32 to Gateway or USB Companion
0x10	SERVICE_REQUEST	Gateway or USB Companion to STM32
0x11	SERVICE_RESPONSE	STM32 to Gateway or USB Companion
0x12	EVENT	STM32 to Gateway or USB Companion

3.3.1.3. Telemetry payloads

HEARTBEAT has an empty payload.

STATUS is 33 bytes:

```
bms_state:u8 | hv_state:u8 | flags:u16 | slave_count:u8 |
bms_active_errors:u32 | bms_latched_errors:u32 |
hv_active_errors:u32 | hv_latched_errors:u32 |
warnings:u32 | uptime_ms:u32 | soc_last_calibration_unix_s:u32
```

Status flag bits are: 0 BMS HV-ready, 1 charging allowed, 2 run request asserted, 3 complete measurements fresh, 4 isolated-UART Gateway peer alive, and 5 automatic balancing enabled. USB Companion heartbeats do not affect bit 4. Bit 6 is SoC valid, bit 7 current sensing enabled, and bit 8 the SoC calibration UTC value valid; bits 9–15 are zero. `slave_count` comes from the configured BMS monitor chain and is variable between builds. A production module contains eight monitor slaves, but a single-slave development chain is supported during bench work. The Gateway must use the reported value and must not reject an odd count.

`uptime_ms` is the STM32 HAL millisecond tick since its most recent reset. It is diagnostic telemetry only, resets to zero after a controller restart, and wraps naturally after about 49.7 days. The STM32 sends STATUS at least every 500 ms while its UART service runs.

PACK is 24 bytes:

```
pack_voltage_uV:u32 | pack_current_raw:i16 | soc_raw:u16 |
min_cell_uV:u32 | max_cell_uV:u32 |
min_ntc_raw:u16 | max_ntc_raw:u16 | min_ic_raw:u16 | max_ic_raw:u16
```

Voltage values are microvolts. Current is signed amperes times 64; positive current is charging. SoC uses the existing BCC raw range ($0 = -100\%$, $65535 = 200\%$), so percent is $100 * (\text{soc_raw} / 65535 * 3 - 1)$. The STM32 bounds its retained estimate and reported SoC to 0–100%, so the wider transport representation is not used in normal operation. NTC raw values convert as $\text{raw} / 65535 * 120 - 20$ degrees C. IC raw values are centikelvin, so degrees C are $\text{raw} / 100 - 273.15$.

Use `pack_current_raw` only when STATUS bit 7 is set and `soc_raw` only when bit 6 is set. SoC is informational and never commands HV. The estimate is retained in the CR2032-powered RTC backup domain with a version marker and checksum; an erased or corrupt value is invalid. Current sensing is deliberately disabled in the single-slave development configuration, so current and SoC are unavailable there. Production enables it only on CID 1. Full-charge calibration requires fresh measurements with no active BMS error, every cell at or above 3.450 V, current from -0.100 A through C/50 (6.28 A for the 314 Ah default), continuously for 300 s. Any failed condition restarts the timer; calibration sets SoC to 100%. The 3.500 V-per-cell inverter charge target deliberately remains above the 3.450 V minimum-cell qualification threshold, so calibration stays reachable while maximum-cell current derating and balancing prevent normal charging from reaching the 3.600 V hard trip. Coulomb counting is bounded at 0% and 100%, allowing a subsequent discharge to lower SoC normally without retaining an out-of-range value. When valid UTC is available, the STM32 persists that instant in integrity-protected backup registers and reports it in `soc_last_calibration_unix_s`; it otherwise reports no calibration time.

HV_VOLTAGES is 12 bytes:

```
valid_flags:u8 | reserved:u8[3] | bat_plus_uV:u32 | load_plus_uV:u32
```

Bit 0 of `valid_flags` means both values came from one fresh, coherent STM32 ADC/DMA scan with a valid ADC reference. Bits 1–7 and the three reserved bytes are zero. The values are positive microvolts from the isolated AMC3330 battery-side (BAT+) and load-side (LOAD+) channels. When the valid bit is clear, consumers show the values as unavailable rather than treating the zero payload as a measurement. These diagnostic readings are sent with every normal fresh snapshot and do not change PCC thresholds, fault decisions, or STM32 safety ownership.

CELL is 51 bytes, once per configured slave:

slave_index:u8 | balance_mask:u16 | cell_voltage_uV[12]:u32

slave_index is zero-based. module_index = slave_index >> 1 and slave_in_module = slave_index & 1. Balance-mask bit 0 represents cell 0 through bit 11 for cell 11; bits 12–15 are zero. Each slave always has twelve cells. In a single-slave development chain, only index 0 is sent; it maps to module 0, slave 0, and no partner frame is implied. balance_mask reports the cells selected for the current balancing pulse after the BCC driver-status check succeeds. It remains stable across brief measurement pauses and therefore does not represent the instantaneous state of the bleed switches.

TEMPERATURE is 11 bytes, once per configured slave:

slave_index:u8 | ntc_raw[4]:u16 | ic_temp_raw:u16

The four NTC values and one IC value are included together, making this packet fixed-size. This matches the installed four-NTC-per-slave configuration and must be enforced as a compile-time configuration check in both endpoint implementations.

The STM32 sends STATUS and a complete HV_VOLTAGES/PACK/CELL/TEMPERATURE snapshot when fresh measurement data is available, no more often than once every 500 ms. It sends EVENT immediately on a change. The Gateway must treat HV_VOLTAGES, PACK, CELL, and TEMPERATURE data as invalid while STATUS says measurements are not fresh.

3.3.1.4. States, faults, and events

BMS states are 0 STARTING, 1 READY, 2 RUNNING, 3 ERROR, and 4 CRITICAL. HV states are 0 OFF, 1 SELF_TEST, 2 PRECHARGE, 3 CONTACTOR_CLOSE, and 4 RUN. CRITICAL has no bitmap: it represents an unrecoverable condition in the current software and forces HV safe-off for the remainder of that boot.

BMS ERROR bitmap bits are: 0 CONFIGURATION_INVALID, 1 SLAVE_UNAVAILABLE, 2 BCC_DIAGNOSTICS, 3 CELL_VOLTAGE_LIMIT, 4 THERMAL_LIMIT, 5 CURRENT_LIMIT, 6 BCC_INTEGRITY, 7 ADC_FAULT, 8 BALANCING_HARDWARE_FAULT, and 9 BCC_COMMUNICATION. Bits 10–31 are reserved and zero.

HV ERROR bitmap bits are: 0 HV_SENSOR_DIAGNOSTIC, 1 BATTERY_VOLTAGE_MISMATCH, 2 LOAD_SIDE_ENERGISED, 3 PRECHARGE_TIMEOUT, 4 PRECHARGE_VOLTAGE_LOST, and 5 CONTACTOR_VOLTAGE_LOST. Bits 6–31 are reserved and zero.

warnings is non-latched. Bits are: 0 WATCHDOG_RESET, 1 STARTUP_DIAGNOSTICS_BYPASSED, and 2 BATTERY_VOLTAGE_MISMATCH_OFF. Bits 3–31 are reserved and zero. WATCHDOG_RESET is a recorded warning that an accepted acknowledgement clears. STARTUP_DIAGNOSTICS_BYPASSED remains present while startup diagnostics are disabled in runtime configuration; BATTERY_VOLTAGE_MISMATCH_OFF remains present while the measurements disagree. WATCHDOG_RESET is set when this STM32 boot followed an independent or window watchdog reset. Gateway liveness, CAN condition, near-limit indication, and disabled automatic balancing are not BMS warnings in this release.

The STM32 compares BAT+ and the slave-reported pack voltage continuously. A disagreement is an OFF-state warning while no run request is present; a run request promotes the same condition to the blocking BATTERY_VOLTAGE_MISMATCH HV error. Releasing the request deasserts that active HV error, retains its latch for acknowledgement, and restores the warning until the readings agree.

An active ERROR is present now. A latched ERROR remains blocking after its live condition clears until the STM32 accepts acknowledgement. The STM32 Fault Manager is the sole owner of active/latched aggregation, acknowledgement, and immediate HV safe-off. Detection modules may only assert or deassert their assigned condition. Startup monitor-chain, register-initialisation, and diagnostics attempts use their source retry budget before asserting their ERROR input. Once exhausted, they use these ordinary ERROR semantics.

A live TPL/BCC communication loss immediately safe-offs HV and re-enters the same non-blocking TPL, CID, register, and measurement initialisation path. A successful complete measurement and fault-status cycle deasserts the active communication error; its latched error still requires acknowledgement.

The independent watchdog starts after the required clock and DMA setup, before every fallible peripheral initialisation, has a nominal 500 ms timeout, and is refreshed every 100 ms only while both the PCC loop and BCC task continue to make progress. BCC initialisation retry waits are non-blocking, so they do not mask a stalled task. A subsequent platform-initialisation failure directly disables the precharge output and contactor PWM before the watchdog reset; the contactor driver pull-down is the hardware fallback. A clock-initialisation failure may occur before the watchdog is available; the hardware pull-down still holds the contactor control line safe.

The STM32 ADC is calibrated at startup. TIM7 triggers a three-rank HV scan at 50 Hz: load-side differential voltage, battery-side differential voltage, and VREFINT. Each rank uses 640.5 ADC clock cycles of acquisition and 256x oversampling with right-shift 4. DMA publishes complete scans as one snapshot. The STM32 drives VREF+ from its 2.9 V internal reference buffer. ADC/DMA errors and a 2.8–3.0 V ADC reference are checked every 20 ms. DMA progress is stale only after no completed scan for 100 ms. Three consecutive failed checks assert ADC_FAULT and immediately safe-off HV. Calibration is not repeated during normal operation.

EVENT is five bytes:

event_id:u8 | value:u32

ID	Event	Value
0x01	BMS state changed	new BMS state
0x02	HV state changed	new HV state
0x03	BMS active errors changed	new active mask
0x04	BMS latched errors changed	new latched mask
0x05	HV active errors changed	new active mask
0x06	HV latched errors changed	new latched mask
0x07	warnings changed	new warning mask
0x08	measurement freshness changed	0 or 1

EVENT is a convenience notification; STATUS is authoritative, so loss of an event is harmless.

3.3.1.5. Services

The service-request payload is service_id:u8 | arguments...; the service-response payload is service_id:u8 | result:u8 | response_data... Result values are 0 OK, 1 DENIED, 2 INVALID, 3 BUSY, and 4 USB_HOST_ACTIVE. INVALID covers an unknown service, bad request length, bad argument, or nonexistent slave index. DENIED means a valid request cannot safely be performed in the current STM32 state. BUSY means another named service is still in progress. USB_HOST_ACTIVE means an enumerated USB host prevents the STM32 ROM bootloader from accepting USART1. OK means that the STM32 accepted and invoked the requested operation; STATUS and EVENT show the resulting operating state.

ID	Service	Request arguments	OK response data
0x01	GET_STATUS	none	33-byte STATUS
0x02	SET_RUN_REQUEST	requested:u8 (0 or 1)	none
0x03	ACKNOWLEDGE_FAULTS	none	none
0x04	READ_REGISTER	slave_index:u8, register:u8	slave_index:u8, register:u8, value:u16
0x05	SET_RTC	unix_time_s:u32 UTC	none
0x06	GET_DEVICE_INFO	none	firmware_version:u32
0x07	PREPARE_STM32_BOOTLOADER	firmware_version:u32, image_length:u32, image_crc32:u32	none
0x08	GET_RTC	none	unix_time_s:u32 UTC
0x09	SET_BALANCING_ENABLED	enabled:u8 (0 or 1)	none
0x0A	COMMIT_STM32_BOOTLOADER	none	none (successful request enters ROM immediately)
0x0B	GET_CONFIG	none	18-byte runtime configuration status and values
0x0C	SET_CONFIG	slave_count:u8, current_sense_slave:u8, shunt_resistance_uohm:u32, battery_capacity_mah:u32, invert_current:u8, balance_enabled:u8, startup_diagnostics:u8	none; the STM32 resets after responding OK
0x0D	GET_DIAGNOSTIC_REPORT	slave_index:u8	latest startup diagnostic result for that BCC

SET_RUN_REQUEST(0) immediately removes the request through the STM32 PCC path. A request of 1 does not guarantee an HV start: the STM32 alone decides whether BMS and HV checks permit the sequence. ACKNOWLEDGE_FAULTS

is denied while any ERROR condition remains active or the BMS is CRITICAL. It is also denied when no inactive BMS/HV ERROR latch or recorded WATCHDOG_RESET warning remains to clear. Accepted acknowledgement clears those states only, leaving live and configuration warnings visible until their underlying condition changes; it deliberately preserves the run request. PCC can therefore restart from OFF immediately once the Fault Manager permits it. Neither the Gateway nor Home Assistant can bypass a fault, write BCC registers, or override the HV supervisor. `firmware_version` is packed as `major | (minor << 8) | (patch << 16) | (build << 24)`.

`SET_BALANCING_ENABLED` controls the STM32-owned automatic balancing enable gate and persists the value in runtime configuration. It applies immediately after the flash write succeeds and does not reboot the STM32. An enabled value does not guarantee that any cell is balancing: the STM32 still requires a running, fault-free BMS, fresh measurements, and its configured cell-voltage and current thresholds. For the MB31 pack, top balancing starts at 3.400 V and a 10 mV difference from the pack-wide minimum cell. A selected cell continues down to a 5 mV difference, providing hysteresis. Balancing is permitted only from -0.100 A through C/10 (31.4 A at 314 Ah), so meaningful discharge inhibits it. Thirty-second pulses are used and all 12 connected cells per slave may be selected at once. The highest-voltage eligible cells receive priority; rotation is used only to share equal-voltage ties. With all channels active, one module board dissipates approximately 6.2 W in its bleed resistors. Before each pulse, the STM32 disables the global drivers, clears every retained CB channel, programs the complete desired mask, and enables the global drivers last. It then verifies the actual driver-status mask. Leaving the running/eligible state disables balancing immediately. A disabled value inhibits the BCC balancing drivers on the next BCC loop. New configuration defaults enable balancing. The configuration schema version is 3. Version-2 records are accepted with startup diagnostics enabled as the safe legacy default and are upgraded to version 3 when next saved through `SET_CONFIG`. `startup_diagnostics` defaults to 1; setting it to 0 skips only the startup BCC diagnostics routine for debugging or bench testing, while all other measurement, fault, balancing, and HV safety checks remain active.

`GET_DIAGNOSTIC_REPORT` returns the slave index, BCC CID, failed-check bitmask, low-level status code, and the diagnostic that was executing when a driver failure occurred. The latest report is held in STM32 RAM until the next startup diagnostic run.

`PREPARE_STM32_BOOTLOADER` and `COMMIT_STM32_BOOTLOADER` form the STM32-update handoff. The Gateway must already have staged and CRC-checked the image. The STM32 validates the prepare request shape and image length; `firmware_version` and `image_crc32` identify the Gateway-staged image but cannot be verified by an STM32 that does not receive its bytes. It returns DENIED unless the run request is off and it can de-energise the HV path. It returns `USB_HOST_ACTIVE` when its USB CDC device is enumerated. Prepare immediately holds HV safely off and blocks normal services, then returns OK. The prepared state expires after 3 seconds unless the Gateway sends commit. Commit has no response: after successfully transmitting it, the Gateway owns USART1 and immediately changes to ROM settings. If prepare or commit transmission fails, the STM32 remains in (or returns to) normal BMS operation; the Gateway reports failure rather than waiting indefinitely. It then performs ROM bootloader sync, transfer, readback verification, and Go directly on USART1; these are not FlexBMS UART messages. On return, the Gateway waits for an application heartbeat before reporting completion. Any failure after erase has begun requires wired STM32 recovery; the Gateway rejects further STM32 OTA attempts until it observes an application heartbeat. There are deliberately no update-data, retry, or status frame types.

3.3.1.6. Test vectors

The following complete frames are hexadecimal, including the little-endian CRC:

HEARTBEAT

46 42 01 01 00 00 00 8F 7A FB 7D

GET_STATUS, sequence 0x2A

46 42 01 10 2A 01 00 01 8F 21 B2 4B

SET_RUN_REQUEST(true), sequence 0x2B

46 42 01 10 2B 02 00 02 01 16 26 B1 DC

READ_REGISTER response, sequence 0x2C, slave 3, register 0x20, value 0x1234

46 42 01 11 2C 06 00 04 00 03 20 34 12 1A 45 F6 9C

PREPARE_STM32_BOOTLOADER, sequence 0x2D, version 1.2.3 build 4,

image length 131072 bytes, image CRC32 0xA1B2C3D4

46 42 01 10 2D 0D 00 07 01 02 03 04 00 00 02 00 D4 C3 B2 A1 42 C9 9E 95

```

COMMIT_STM32_BOOTLOADER, sequence 0x2E
46 42 01 10 2E 01 00 0A 50 6F 02 53

```

3.3.2. Pinout

The Gateway uses an ESP32-C3-WROOM-02U-N4. The module pin names and numbers below follow the Espressif datasheet; net names follow the Master schematic. GPIO17 is used internally for module flash and is not exposed.

Module pin	GPIO / module name	Net	Use
1	3V3	p3v3_INV	3.3 V supply
2	EN	EN	Chip enable
3	GPIO4 / IO4	CAN_TX	CAN controller transmit
4	GPIO5 / IO5	CAN_RX	CAN controller receive
5	GPIO6 / IO6	RS485_R0	RS-485 receiver output
6	GPIO7 / IO7	RS485_DI	RS-485 driver input
7	GPIO8 / IO8	I08/STRAP	Strapping pin
8	GPIO9 / IO9	I09/BOOT	Boot-mode strap
9; EPAD 19	GND	GND_INV	Ground and module ground pad
10	GPIO10 / IO10	ESP_VBUS_SENSE	USB VBUS sense
11	GPIO20 / RXD	—	Not connected; UART0 RX unused
12	GPIO21 / TXD	—	Not connected; UART0 TX unused
13	GPIO18 / IO18	USB_ESP32_N	USB D-
14	GPIO19 / IO19	USB_ESP32_P	USB D+
15	GPIO3 / IO3	UART_ESP_RX	STM32 USART1 TX input
16	GPIO2 / IO2	UART_ESP_TX	STM32 USART1 RX output; strapping pin
17	GPIO1 / IO1	USR_LED	Gateway user/status LED
18	GPIO0 / IO0	RS485_EN	RS-485 driver enable

3.4. Energy management system

PLANNED

The EMS is the supervisory optimization layer. It should combine battery availability with site demand, PV production, inverter limits, dynamic electricity prices, EV charging, and other flexible loads.

The EMS may request charging or discharging, but it must operate inside limits supplied by the BMS and inverter. It must tolerate missing telemetry and fall back to a safe, predictable operating policy.

The EMS implementation and deployment location are not yet defined in this repository. Its first interface should be a small, explicit contract containing battery state, available charge/discharge power, requested power, data freshness, and reason/status information.

3.5. Home Assistant integration

CURRENT

Home Assistant is the intended user-facing integration point for the installation. It can provide:

- battery, inverter, PV, grid, and load dashboards;
- fault and availability notifications;
- manual operating-mode selection;
- automation around dynamic prices and flexible loads; and
- long-term energy and temperature history.

Home Assistant must not become a safety dependency. If Home Assistant is unavailable, the STM32 must continue to protect the battery and the EMS must remain within the last valid limits or stop requesting power.

The Gateway publishes one retained MQTT Device Discovery record and retained compact BMS state below a MAC-derived `flexbms/flexbms_XXYYZZ` prefix. It exposes pack voltage/current/power, SoC when valid, cell and temperature extrema, BMS/HV state, active faults, freshness, UART health, and the STM32-owned run-request switch. Minimum and maximum cell voltage display in V with three decimals; cell delta displays as whole mV. It also exposes separate diagnostic uptime values for the ESP32 Gateway and STM32 BMS controller. ON and OFF requests share the existing single Gateway-to-STM32 service slot; the next STM32 status is authoritative and never promises contactor closure. Broker settings (host, port, username, write-only password) are configured only through the trusted station-LAN Companion page. The binary freshness, run-state, fault, and UART-health entities publish canonical MQTT ON/OFF states. Fault acknowledgement, balancing, firmware upload, registers, raw diagnostics, and inverter/EMS power control remain outside Home Assistant MQTT v1.

An optional Gateway-local RS485 extension reads zero, one, or two SDM72D-M-2 meters. Both slots are disabled by default. The Companion configures each meter's name, slave address, and reverse power direction, plus shared baud rate (default 9600), parity, and stop bits. UART0 uses GPIO7 TX, GPIO6 RX, and GPIO0 driver enable independently of the STM32 UART1 link. The Gateway reads phase currents, phase and total signed active power every second, and import/export kWh every 30 seconds. Each enabled slot gets a separate HA device with stable Gateway-and-slot identity. Failed or stale reads become unavailable rather than zero; non-retained electrical and energy samples expire independently. Reversing direction negates active power and exchanges import/export energy while preserving current magnitudes. The extension performs no inverter regulation; Home Assistant owns that control. Settings persist in Gateway NVS and are editable only on the trusted station LAN. Register offsets, wiring, freshness, and commissioning checks are documented in `architecture/rs485-energy-meters.md`.

3.6. Companion application

CURRENT

Software/Companion is the FlexBMS-owned BMS-only application with two web targets: a direct USB Web Serial target for independent service diagnostics and an ESP32-hosted WebSocket target for local maintenance. A portable, unsigned Windows executable packages the direct-USB target in a thin Electron/Chromium wrapper. It has no installer, updater, or native serial-protocol implementation.

Both targets provide stale-status explanation, live fresh telemetry, CSV logging, named service requests, device-time readback, and read-only register inspection. The Gateway obtains UTC from NTP and shows its latest successful RTC sync in the browser locale; direct USB shows that NTP status is Gateway-only. The Gateway target also shows Wi-Fi and UART state and, only in connected station mode, single-bundle firmware updates. The direct-USB target exposes neither a raw terminal nor firmware-update UI; Gateway requests use an explicit allowlist.

`scripts/build-release.ps1` interactively confirms the release version, updates the Companion package version when a new one is entered, and builds the Companion, Gateway, and STM32 artifacts with checksums and one `FlexBMS_bundle.fbu` OTA package containing both targets, versions, byte lengths, and CRC-32 values. `-Version` supports unattended use. Each release includes `flash-release.ps1`: ST-Link/SWD flashes the STM32 image, while a selected ESP32-C3 COM port flashes a complete Gateway factory image. Both actions start immediately after checksum verification. Repeating an existing version requires an explicit choice and creates a timestamped output without overwriting the prior release.

4. Operating concept

4.1. Start-up

At start-up, the STM32 validates its configuration and initializes the complete TPL chain. It assigns each battery monitor a chain address, configures its registers, runs diagnostics, and obtains a fresh measurement set. Both HV outputs remain off. After BMS validation, the separate HV supervisor can run its self-test and precharge sequence only when a new run request is supplied.

4.2. Normal operation

During normal operation the STM32 repeatedly measures the pack and evaluates its safety state. Battery limits and status are made available to the surrounding system. The inverter converts energy between the DC battery and the AC installation, while the future EMS decides the desired operating point.

4.3. Faulted operation

A battery, measurement, or communication fault immediately removes relay permission. Monitoring continues where possible so that the live condition can be observed. Faults remain latched until they are acknowledged through the supported service path and the BMS has successfully revalidated the battery chain.

The BMS exposes the HV supervisor as one aggregate blocking fault while the supervisor retains separate active, latched, and historical HV reasons. The Companion clear command can clear an HV latch only when the live HV conditions are healthy. The BMS then repeats chain initialization, diagnostics, and a complete fault-checked measurement cycle before the latch is released. If the retained run request is true, the BMS may then resume its normal startup sequence. Historical reasons remain available until reboot.

Configuration and initial TPL setup faults require a reboot. The ESP32, EMS, and Home Assistant cannot override a blocking STM32 fault.

5. Interfaces and ownership

Information or action	Owner	Consumer
Cell voltages and temperatures	STM32 BMS	Diagnostics, inverter limits, telemetry
Battery current and SoC	STM32 BMS	Inverter, EMS, Home Assistant
Battery safety limits	STM32 BMS	Inverter and EMS
Contactor and precharge	STM32 BMS	Physical HV path
Inverter operating state	Inverter	ESP32 / EMS / Home Assistant
Requested charge/discharge power	EMS	Inverter-facing control interface
Dashboards and automation	Home Assistant	User and household services

The protocol between these layers should carry explicit validity and freshness information. A stale value must never be interpreted as a current permission to charge, discharge, or close the contactor.

6. Current project structure

Repository area	Contents
Hardware/Master	Master schematic, PCB, harnesses, and production data.
Hardware/ModuleBoard	Battery module-board schematic, PCB, and production data.
Software/Master	STM32G491 firmware and platform configuration.
Software/Companion	Current FlexBMS BMS maintenance UI with direct USB, Gateway, and portable Windows package outputs.
Software/Gateway	ESP32 UART v1, Wi-Fi provisioning/recovery, compiled Companion serving, Home Assistant MQTT discovery, optional RS485 energy meters, and OTA; CAN observation remains planned.
Documentation/protocol/uart-v1.md	Canonical framed BMS protocol for Gateway UART and direct USB CDC, including test vectors.
Simulations	Electrical simulation files used during hardware development.
Documentation	This system overview and its Typst build setup.

The repository contains the initial ESP32 Gateway and FlexBMS Companion implementations. EMS and Home Assistant remain planned.

7. Open topics

This overview is intentionally incomplete. The next useful documentation additions are:

- confirmed as-built module, NTC, and high-voltage topology;
- final inverter selection, protocol compatibility, and commissioning sequence;
- system-level operating, fault, recovery, and shutdown policy;
- commissioning of the ESP32 telemetry, local-network, and update architecture; and
- EMS and Home Assistant interfaces, fallback behavior, and user permissions.